

FREE PLAYBOOK

The Compliant Tracking Stack

For doctors, lawyers, and home service.
Tracking that holds up.

INSIDE THIS PLAYBOOK

GA4

GTM

Meta CAPI

Google Ads

TikTok

CallRail

Consent Mode v2

GSC API

AI BrandFactory

For doctors, lawyers, and home service businesses. Tracking that holds up under HIPAA, attorney advertising rules, and consent law.

Who this is for

Three vertical groups, one shared problem.

- **Medical practices, clinics, DPC, dental, behavioral health.** You face HIPAA on the back end and Meta's Health and Wellness account restrictions on the front end. The standard tracking guides will get you sued or get your ad account flagged.
- **Law firms, solo attorneys, multi-state practices.** State bar advertising rules vary, attorney-client privilege constrains what you can store about a lead, and form data on a contact page can become a discovery target. Generic conversion tracking ignores all of this.
- **Home service businesses, contractors, trades, multi-location service brands.** Most of your conversions are phone calls, not form submits. Most of your ad budget rides on attribution that gets dropped at the form because the lead form has no UTM capture and the call is untagged.

The technical stack is the same across all three. The compliance settings, the event names, the data stripping, the retention rules differ. This playbook covers both.

What is in this playbook

Six parts plus a companion git repository with working code templates.

- **Part 1, Why Tracking Got Hard in 2026.** Consent Mode v2, third-party cookie sunset, Meta health restrictions, vertical compliance rules.
- **Part 2, The Foundation.** GA4 plus Google Tag Manager plus Consent Mode v2 plus a CMP that actually gates third-party scripts.
- **Part 3, Server-Side Conversions.** Meta CAPI plus Google Ads Enhanced Conversions, with the SHA-256 hashing pattern, the eventID dedup logic, and the data you must never send.
- **Part 4, The Vertical Layer.** Doctor pattern (HIPAA, BAA-signed CDP, LDU flag), lawyer pattern (state bar advertising rules, privileged form data), home service pattern (call-driven attribution, service-area tracking).

- **Part 5, Call Tracking and Lead Attribution.** CallRail dynamic number insertion, first-party UTM capture middleware, the lead_attribution table, and the Google Search Console API for organic keyword visibility.
- **Part 6, Validation, Deployment, and the 8 Failures.** Tag Assistant, Meta Events Manager Test Events, GTM preview mode, the pre-launch checklist, the 8 mistakes that silently break tracking.

The companion git repository ([AI-BrandFactory/professional-tracking-stack](#)) ships working Next.js 16 component templates, server-side route handlers for every API integration in this playbook, an `env.example` covering all variables, and three vertical-specific configuration files (doctor, lawyer, home service).

Part 1, Why Tracking Got Hard in 2026

Conversion tracking quietly broke for professional services across 2024, 2025, and 2026. Most practices and firms still run the setup they deployed two or three years ago and assume it is working. It is not. Here is what changed and what it costs.

Change 1, Consent Mode v2 became mandatory in the EEA (March 2024)

Google's Consent Mode v2 enforces that GA4 and Google Ads tags must receive a structured consent signal before they can use any user data for measurement or advertising. The signal has four flags:

- `analytics_storage`
- `ad_storage`
- `ad_user_data`
- `ad_personalization`

Each must be set to `granted` or `denied` based on the visitor's actual consent. If your tags load without these signals (or if they default to `granted` without consent), Google's enforcement path is to discard the data and (for Google Ads) limit campaign optimization.

The enforcement window started March 2024 in the EEA, the UK, and Switzerland. By 2026 it is fully active and being mirrored by similar requirements in Quebec (Law 25), California (CPRA), and Brazil (LGPD). If your practice has any visitors from these regions and your CMP does not gate scripts via Consent Mode v2, you are losing measurement data and potentially exposing yourself to regulatory action.

Change 2, Third-party cookies are sunseting in Chrome (2024 to 2026)

Chrome started phasing out third-party cookies in early 2024 with a small percentage of traffic, expanding through 2025. By the time you read this in April 2026, the practical reality is that third-party cookies are unreliable in Chrome and useless in Safari and Firefox, both of which blocked them earlier.

This breaks:

- **Cross-site retargeting** without server-side fallback (Meta CAPI, Google Ads Enhanced Conversions).
- **View-through attribution** in most ad platforms.
- **Visitor identification across subdomains** unless you use first-party cookies set on a parent domain.

The fix is server-side conversions plus first-party data capture. We cover both in Parts 2 and 3.

Change 3, Meta restricted lower-funnel events for healthcare (late 2025)

Starting late October 2025, Meta began restricting bottom-of-funnel conversion events (Lead, Schedule, Purchase, CompleteRegistration) for ad accounts categorized under Health and Wellness.

This affects:

- Direct primary care practices
- Behavioral health and mental health providers
- Dental practices
- Medical spas and aesthetics
- Functional and longevity medicine
- Any clinic running ads on Meta where the page subject is a health condition or treatment

For affected accounts, optimization is restricted to top-of-funnel objectives (Awareness, Traffic, Engagement). Lead optimization may still work via server-side CAPI through a Business Associate Agreement-signed CDP, but the configuration is strict. Sending health-related URL paths, condition-specific event names, or Protected Health Information will get you flagged and can result in account suspension.

Behavioral health providers face the strictest enforcement. If you fall into that category, the playbook in Part 4 walks through what is still possible.

Change 4, Attorney advertising rules vary state by state and apply to digital tracking

Most state bars have advertising rules that govern what attorneys can communicate publicly. In 2024 and 2025, multiple state bars issued ethics opinions clarifying that:

- Form fields that capture case-type data (DUI, divorce, personal injury) become subject to advertising restrictions in some states once stored
- Lead-tracking pixels that fire on contact form submission with the case type as an event parameter can violate solicitation rules
- Attorney-client privilege may attach to information in a contact form even before formal representation, depending on the state

The result: a law firm cannot run the same Meta Pixel + Lead-event setup as a SaaS company. Part 4 covers the lawyer-specific adjustments.

Change 5, Lead attribution silently lost to form abandonment and call tracking gaps

Across all professional services, but especially home service businesses, the pattern is the same:

- A visitor lands on a paid ad with `gclid` in the URL
- They browse, then call the phone number on the page
- The call is recorded in CallRail or a generic phone tracker
- The lead is logged in the CRM
- But the `gclid` was never captured and never associated with the call
- The campaign that generated the lead gets zero attribution credit

For home service businesses where 60 to 70 percent of leads come by phone, this means the entire ad spend bidding model is operating on bad data. Part 5 covers how to fix it with CallRail dynamic number insertion plus first-party UTM capture middleware.

What it costs you

The five changes above combine into a specific, measurable cost:

- **Lost data.** GA4 and Meta both report fewer conversions than actually happened. The platforms then push spending toward audiences that match the data they have, not the data they should have.

- **Wasted ad spend.** Google Ads Smart Bidding and Meta Advantage+ both use conversion data as the bidding signal. Bad signal, bad spend.
- **Compliance exposure.** Healthcare and legal verticals face actual regulatory action for tracking misconfigurations. The FTC has fined practices for sending health data to third parties via Meta Pixel. The 2023 GoodRx case was the template; multiple state attorneys general have followed.
- **Attribution blindness.** Without first-party data and call tracking integration, you cannot answer "which campaign drove this revenue" for the leads that did convert. Decisions get made on intuition instead of data.

The rest of this playbook is the fix. Each part builds on the previous one. Part 2 sets the foundation. Parts 3 through 5 add the layers. Part 6 validates and deploys.

Part 2, The Foundation: GA4 Plus GTM Plus Consent Mode v2

Three components form the base layer for every professional services site in 2026:

1. **Google Analytics 4** as the analytics engine
2. **Google Tag Manager** as the tag orchestration layer (optional but recommended)
3. **A consent management platform** that gates third-party scripts via Consent Mode v2

The order in which these load matters. The default state matters. The categories matter. Get any of these wrong and the rest of the stack does not work.

Decision 1, GA4 directly or GA4 via GTM?

Both are valid in 2026. The right choice depends on how many tracking tools you plan to run.

Use GA4 directly when:

- You only need GA4, no Google Ads conversion tag, no Meta Pixel, no third-party tags
- You have a developer who is comfortable editing site code for every tag change
- You want the lowest possible script weight

Use GTM when:

- You will run two or more tracking tools (GA4 plus Google Ads plus Meta Pixel plus CallRail, for example)
- You want non-developers (the marketing person, the agency) to make tag changes without site code edits
- You want a single point to manage consent gating across every tag

For a typical professional services practice that runs Google Ads plus Meta plus call tracking plus organic SEO measurement, GTM is the right answer. The container becomes the central control point and adding a new tag (TikTok, LinkedIn Ads, a new conversion goal) is a 5-minute change in the GTM UI rather than a code deploy.

Decision 2, Consent Mode v2 default state

Before any tracking script loads, the page must initialize the Google consent dataLayer with default values. For a site that serves any visitors from the EEA, UK, Switzerland, California, Quebec, or Brazil (which is most professional services sites), the default must be **denied** for all four flags.

```

window.dataLayer = window.dataLayer || [];
function gtag() { dataLayer.push(arguments); }

gtag('consent', 'default', {
  'analytics_storage': 'denied',
  'ad_storage': 'denied',
  'ad_user_data': 'denied',
  'ad_personalization': 'denied',
  'wait_for_update': 500
});

gtag('js', new Date());

```

Three rules:

- 1. This script must run before any other tracking script loads.** In Next.js with the App Router, place it inside `<Script>` with `strategy="beforeInteractive"`. In a static site, it goes in `<head>` before any GA4 or GTM tag.
- 2. `wait_for_update: 500`** tells Google to hold tag firing for up to 500ms while waiting for the consent decision. This is the window your CMP must update consent within.
- 3. Every flag must be either `granted` or `denied`.** Leaving any flag undefined is treated as `denied` but produces less reliable behavior in Search Ads 360 and other Google tools.

Once the visitor accepts cookies in your CMP, the CMP fires a consent update:

```
gtag('consent', 'update', {  
  'analytics_storage': 'granted',  
  'ad_storage': 'granted',  
  'ad_user_data': 'granted',  
  'ad_personalization': 'granted'  
});
```

GA4 and Google Ads will then start firing tracking events. If consent is denied, GA4 still fires events but they are sent in cookieless mode (anonymous, no user-level data), which gives you basic page-view counts without the ability to attribute or retarget.

Decision 3, Which CMP to install

Three options, ranked.

Option A, vanilla-cookieconsent v3 (recommended for full control)

Open-source, npm-installable, no subscription, full IAB TCF v2 support, predictable consent change events, and easy to integrate with Consent Mode v2.

```
npm install vanilla-cookieconsent
```

This is the foundation the companion git repo uses. The full configuration with three categories (necessary, analytics, marketing) plus the consent-update wiring is in [lib/cookie-consent/config.ts](#) of the companion repo.

Option B, CookieYes (recommended for non-technical setup)

A managed CMP via CDN script tag. No npm install, full GDPR/CCPA/LGPD coverage, branded admin UI for the practice owner. Slightly slower initial paint (extra third-party script). Has a free tier suitable for most professional services sites.

Option C, Cookiebot (recommended for healthcare and legal compliance audits)

The premium option, used by enterprises that need an audit trail of every consent decision. Has the deepest coverage of regional regulations and produces the cleanest compliance reports. Paid only.

For a typical doctor, lawyer, or home service practice: vanilla-cookieconsent v3 if you have a developer, CookieYes if you do not.

Decision 4, Cookie categories for professional services

Three categories cover most professional services sites. A fourth (functional) is optional.

Category	Purpose	Default	Examples
Necessary	Session, auth, CSRF, consent state itself	Always on (cannot be disabled)	Session cookies, CSRF tokens, the consent cookie
Analytics	Anonymous usage measurement	Off	GA4 (when running with consent), Microsoft Clarity, internal analytics
Marketing	Cross-site advertising, retargeting	Off	Google Ads conversion tag, Meta Pixel, TikTok Pixel, LinkedIn Insight Tag
Functional (optional)	Non-essential features	Off	Live chat widget, video embeds, third-party booking widgets

For healthcare specifically, add a paragraph in the cookie banner that clarifies: "We do not share health information with third parties and do not use marketing cookies for health-related personalization." This is a defensive disclosure that helps in any consent-related dispute.

For legal practices, similar language: "Form data submitted on this site is not shared with third-party advertising platforms." Some state bars require this kind of explicit disclosure.

The load order that has to be correct

The order in which scripts initialize on a page is the single most common cause of broken tracking. The correct order:

1. **Consent Mode v2 default** (denied for everything), fires before any tag
2. **CMP loads** and reads any stored consent
3. **If stored consent exists**, CMP fires `consent update` immediately with the stored values
4. **GA4 / GTM loads** and inherits the current consent state

5. **Other tags** (Meta Pixel, Google Ads) load and inherit the current consent state
6. **If no stored consent**, CMP shows the banner and waits for user action
7. **User accepts or rejects**, CMP fires `consent update` with new values
8. **Tags re-evaluate** based on the updated consent

If step 1 happens after step 4, GA4 will fire one or more events with default-granted consent before being told to deny. Those events are GDPR violations.

If step 4 happens before step 3, GA4 will fire its initial PageView with denied consent even when the user has previously accepted. The visitor is now an anonymous bounce.

The companion repo's `app/layout.tsx` sets this order correctly for Next.js 16. For other stacks, the same principle applies: consent default first, CMP next, all third-party scripts after.

What the foundation gives you

When the foundation is correct:

- GA4 captures page views, sessions, and engagement events with consent-aware data
- Google Ads can run conversion tracking without violating EEA / California requirements
- Every other tag (Meta, TikTok, CallRail) inherits the same consent gating automatically when added through GTM
- The CMP shows a clear banner with three categories and the visitor controls what runs
- Consent decisions are logged with timestamps in the CMP, providing the audit trail regulators ask for

This is the baseline. Parts 3 and 4 add server-side conversions and vertical-specific configuration on top of it.

The rest of the AI BrandFactory library builds on patterns like this. See the full set at aibrandfactory.com.

Part 3, Server-Side Conversions: Meta CAPI Plus Google Ads Enhanced Conversions

Browser-side pixels are no longer reliable in 2026. Three forces broke them:

- **Third-party cookies are gone or going.** Safari blocked them in 2020. Firefox in 2019. Chrome started phasing them out in 2024 and they are unreliable by 2026.
- **Browser-level tracking prevention** strips fingerprinting signals. iOS 14.5+ requires App Tracking Transparency for any cross-app tracking. Safari's Intelligent Tracking Prevention purges first-party cookies set by JavaScript after 7 days.
- **Ad blockers** block Pixel and similar tags outright. Adoption is at 30 to 40 percent in some demographics.

The fix is server-side: when a conversion happens, your server (not the browser) sends the event to Meta and Google directly. The connection is server-to-server, so it bypasses ad blockers, cookie restrictions, and tracking prevention. Combined with the browser-side pixel and proper event deduplication, you get the most complete attribution possible in 2026.

Meta Conversions API (CAPI)

Meta CAPI is a server-to-server endpoint that accepts event data via authenticated POST. The minimum required structure for every event:

```

{
  "data": [
    {
      "event_name": "Lead",
      "event_time": 1745491200,
      "action_source": "website",
      "event_source_url": "https://example.com/contact",
      "event_id": "lead_5f3e2a1b9c8d7e6f5a4b3c2d1e0f",
      "user_data": {
        "em": ["sha256-hashed-email"],
        "ph": ["sha256-hashed-phone"],
        "client_ip_address": "203.0.113.5",
        "client_user_agent": "Mozilla/5.0 ...",
        "fbp": "fb.1.1745491100.1234567890",
        "fbclid": "fb.1.1745491100.AbCdEf"
      },
      "custom_data": {
        "value": 0,
        "currency": "USD",
        "content_name": "Contact Form",
        "content_category": "Inquiry"
      }
    }
  ]
}

```

The five fields that matter most for accurate attribution:

event_id for deduplication

The same conversion should be sent from both the browser-side Pixel and the server-side CAPI for maximum reliability. To prevent Meta from counting the same conversion twice, both sides must send the same **event_id**. Generate it on the server when the conversion fires, pass it to the browser, and use it in both pathways.

```
// Server-side: generate the eventID
const eventId = `lead_${crypto.randomUUID()}`;

// Browser-side Pixel: pass eventID in fbq call
fbq('track', 'Lead', {}, { eventId: eventId });

// Server-side CAPI: include same eventID in event_id field
{
  "event_name": "Lead",
  "event_id": eventId,
  // ...
}
```

Meta's matching window is 48 hours. Events with the same `event_id` arriving within that window get merged.

`user_data` hashed identifiers

Meta needs identity signals to match the conversion back to the visitor's Meta account. The four most useful signals:

- `em` : SHA-256 hash of lowercased trimmed email
- `ph` : SHA-256 hash of phone in E.164 format without the + prefix
- `fbp` : First-party Pixel cookie set by Meta (read from the browser, sent server-side)
- `fbclid` : Click identifier from `fbclid` URL parameter (capture on landing, send server-side)

The hashing pattern in pseudocode:

```
const crypto = require('crypto');

function hashEmail(email) {
  return crypto
    .createHash('sha256')
    .update(email.trim().toLowerCase())
    .digest('hex');
}

function hashPhone(phone) {
  // Strip everything except digits, then prepend country code if missing
  const digits = phone.replace(/\D/g, '');
  return crypto
    .createHash('sha256')
    .update(digits)
    .digest('hex');
}
```

Three rules:

- 1. Hash server-side, never client-side.** The whole point is that the raw value never leaves your infrastructure.
- 2. Hash after trim and lowercase.** `John@example.com` and `john@example.com` should produce the same hash.
- 3. Phone numbers must include country code.** Hash `15551234567` , not `5551234567` . Otherwise Meta cannot match.

`client_ip_address` and `client_user_agent`

Both should be captured from the request headers on the server. They help Meta deduplicate against the Pixel-side event and are part of the matching signal.

`fbp` and `fbclid`

The Pixel sets `_fbp` as a first-party cookie when it loads. Read this from the cookie jar on the server and pass it as `fbp` . For `fbclid` , capture `fbclid` on landing in middleware, format it as `fb.1.{timestamp}.{fbclid}` , and persist it for the session.

Limited Data Use (LDU) flag

For any account in a regulated jurisdiction (California, Colorado, Connecticut, Virginia, Utah, plus Brazil and certain healthcare contexts), set the LDU flag on every event:

```
{
  "data": [{ /* event */ }],
  "data_processing_options": ["LDU"],
  "data_processing_options_country": 1,
  "data_processing_options_state": 1000
}
```

This tells Meta: use this event for measurement only, not for personalization or audience targeting. It is the technical signal that maps to "Limited Data Use" in Meta's privacy framework. For healthcare and behavioral health practices, this flag is effectively mandatory.

Google Ads Enhanced Conversions

Google's equivalent of CAPI is Enhanced Conversions for Leads. The mechanism: when a conversion fires (form submit, qualified call, booked appointment), upload the hashed visitor identifiers to Google Ads via the Google Ads API. Google matches the hash against signed-in Google users and credits the conversion to the right ad click.

The API request structure:


```

const customer = client.Customer({
  customer_id: process.env.GOOGLE_ADS_CUSTOMER_ID,
  refresh_token: process.env.GOOGLE_ADS_REFRESH_TOKEN
});

await customer.conversionAdjustmentUploads.uploadConversionAdjustments({
  customer_id: process.env.GOOGLE_ADS_CUSTOMER_ID,
  conversion_adjustments: [{
    conversion_action: `customers/${customerId}/conversionActions/${conversionActionId}`,
    adjustment_type: 'ENHANCEMENT',
    user_identifiers: [
      { hashed_email: hashEmail(email) },
      { hashed_phone_number: hashPhone(phone) }
    ],
    user_agent: req.headers['user-agent'],
    gclid_date_time_pair: {
      gclid: gclidFromForm,
      conversion_date_time: '2026-04-24 14:30:00-08:00'
    }
  }]
});

```

The same hashing rules apply: SHA-256, lowercased, trimmed. The `gclid_date_time_pair` ties the conversion back to the original Google Ads click.

Enhanced Conversions improve match rate by 5 to 10 percent on average, sometimes higher for service businesses where the visitor reads the page on mobile and converts later on desktop.

What you must never send (the data destruction list)

The same rules apply across both Meta CAPI and Google Ads. Sending any of the following gets you flagged, fined, or sued.

Always prohibited

- **Raw email, phone, name, address.** Hash everything before it leaves your server.
- **Social Security numbers, government IDs, financial account numbers.** Even hashed, these have no business in an ad-platform event.
- **Passwords or authentication tokens.** Obvious, but it has happened.

Prohibited in healthcare contexts

- **Health condition data in event names.** Never `booked_diabetes_consultation` . Use `Schedule` with generic `content_category` .
- **Health condition data in URL paths.** Strip or generalize. Send `event_source_url: example.com/services` instead of `example.com/services/cardiac-stress-test` .
- **Diagnostic, treatment, medication, or symptom strings** anywhere in `custom_data` .
- **Inferred PHI from page subject.** A page about "depression treatment" cannot fire a `Lead` event with `content_name: "Depression Lead"` .

The 2023 GoodRx FTC case fined the company \$1.5M for sending health-condition data to Meta and Google via Pixel. Multiple class actions have followed against Novant Health, Advocate Aurora, and others. Healthcare practices that send PHI to ad platforms face actual financial liability, not just policy slaps.

Prohibited in legal contexts

- **Case type or case description data.** Sending `content_category: "DUI Defense"` violates state bar advertising rules in multiple states.
- **Lead form free-text content.** The "tell us about your situation" textarea may be subject to attorney-client privilege depending on jurisdiction. Never send it as `custom_data` .
- **Names of opposing parties or judges** that may appear in form data. Strip with a server-side filter.

Prohibited in home service contexts (lighter but still important)

- **Customer payment data.** PCI-DSS prohibits sending card numbers anywhere. Even hashed.
- **Property address detail** beyond city/state. Especially for security and pest-control businesses where address could compromise client safety.

The "send only the minimum" pattern

The safest server-side event for a professional services site looks like this:

```
{
  "event_name": "Lead",
  "event_time": 1745491200,
  "action_source": "website",
  "event_source_url": "https://example.com/contact",
  "event_id": "lead_<uuid>",
  "user_data": {
    "em": ["<sha256>"],
    "ph": ["<sha256>"],
    "client_ip_address": "<from request>",
    "client_user_agent": "<from request>",
    "fbp": "<from cookie>",
    "fbc": "<from session>"
  },
  "custom_data": {
    "value": 0,
    "currency": "USD",
    "content_name": "Contact",
    "content_category": "Inquiry"
  }
}
```

That is everything an ad platform actually needs. Anything beyond this is either useless for attribution or actively risky.

The companion git repo includes `app/api/track/meta/route.ts` (CAPI handler) and `app/api/track/google-ads/route.ts` (Enhanced Conversions handler) with the hashing utilities, header capture, and the safe-default `custom_data` shape pre-built.

Part 4, The Vertical Layer: Doctor, Lawyer, and Home Service Patterns

The foundation in Parts 2 and 3 is the same for every professional services site. The configuration on top of it differs by vertical. Each section below covers what to add, what to remove, and what to never do for that vertical.

4.1 The Doctor Pattern (Healthcare, Dental, Behavioral Health, Aesthetics)

The most constrained vertical. HIPAA on the back end, Meta's Health and Wellness restrictions on the front end, and FTC enforcement on the data flow between them.

Architecture decision

Run server-side CAPI through a HIPAA-compliant Customer Data Platform (CDP) that has signed a Business Associate Agreement with the practice. Three CDPs offer this in 2026:

- **Freshpaint** (with BAA option, healthcare-specific configuration)
- **Ours Privacy** (purpose-built for HIPAA)
- **Able CDP** (broader CDP with healthcare BAA)

The practice's site fires events to the CDP. The CDP strips PHI, hashes PII, generalizes event names, and forwards to Meta CAPI, Google Ads, and any other ad platform. The practice never touches Meta Pixel directly, and the ad platforms never see raw patient data.

What to remove

1. **Meta Pixel.** Remove the browser-side Pixel completely from healthcare sites. The risk of accidentally firing PHI in a custom event or page-view URL is too high. CAPI through the CDP is the only safe pathway.
2. **Auto-page-view events** from any tag that fires on URL paths containing condition or treatment keywords. Strip the page-view firing from those URLs entirely or generalize the URL before firing.
3. **Form-field tracking** that captures any free-text from patient intake forms. The reason for visit, symptoms, medication list, and insurance details are PHI.
4. **CallRail call recording** unless the practice has signed CallRail's BAA and access to recordings is locked down with role-based controls and audit logging.

Event mapping for healthcare

The standard event names that are safe and useful in 2026:

Action	Event	Notes
Email captured (newsletter signup)	Lead	value: 0 , content_name: "Newsletter"
Contact form submitted	Contact	Strip all form free-text; do not send reason for visit
Appointment booking initiated	InitiateCheckout	Generic, no service detail
Appointment confirmed	Schedule	content_name: "Service Booking" , no service type
New patient enrollment	CompleteRegistration	value: , content_name: "Membership"
Service page view	ViewContent	Generalized URL, generalized content_name

For Meta accounts already classified as Health and Wellness, the Lead, Schedule, and CompleteRegistration events may be unavailable as conversion goals in Ads Manager. Events still fire and arrive in the Meta dashboard, but Meta will not bid toward them as targets. The practice can still use:

- **Awareness and Traffic campaigns** with these events as measurement-only signals
- **Custom Audience retargeting** based on hashed email lists uploaded from the CRM
- **Lookalike Audiences** built from the existing patient list (hashed)

LDU flag is mandatory

Every event sent from a healthcare practice should include `data_processing_options: ["LDU"]` . Even outside California, this signals to Meta that the data is for measurement only.

URL sanitization rule

The CDP (or the server-side handler) must rewrite `event_source_url` before forwarding. The pattern:

```
function sanitizeHealthcareUrl(url) {
  const conditions = [
    'cardiac', 'diabetes', 'depression', 'anxiety', 'cancer',
    'hormone', 'fertility', 'addiction', 'mental-health',
    'stroke', 'arthritis', 'asthma', 'copd', 'hiv'
    // ... extended list
  ];

  let sanitized = url;
  for (const term of conditions) {
    if (sanitized.includes(term)) {
      // Replace condition-specific path with /services
      sanitized = sanitized.replace(/\/services\/[^\/]*/, '/services');
      break;
    }
  }
  return sanitized;
}
```

This is one of the most common compliance failures: the page URL contains the condition, the page-view event fires with the URL, the URL goes to Meta, Meta now knows that visitor was on a depression-treatment page. The CDP or middleware must scrub before any forwarding.

4.2 The Lawyer Pattern (Solo Attorneys, Multi-Practice Firms)

The compliance considerations are different. There is no HIPAA equivalent in the legal world, but state bar advertising rules are the binding constraint, and attorney-client privilege creates downstream data-handling implications.

Architecture decision

Server-side CAPI plus Pixel can both be used by law firms (no platform-level restriction like Health and Wellness). The constraints are about what data you fire, not which channel you use.

What to remove

- 1. Case-type categorization in event data.** Do not send `content_category: "Personal Injury"` or `content_name: "DUI Case"`. Use generic categories like "Inquiry" or "Consultation Request".
- 2. Free-text "tell us about your case" data.** This is potentially privileged. Never log it to ad platforms. Store it in the CRM and treat it with the same care as formal client communications.

- 3. Lead-form pixels that fire based on practice-area page.** A firm with a personal injury landing page should not fire a `Lead` event with the page URL or page name attached.

State bar considerations

Three rules to verify against the state bar of the firm's jurisdiction:

- 1. Solicitation rules.** Some states (Florida, Texas, New York are notable) prohibit certain forms of targeted advertising. Lookalike audiences built from existing client lists may violate solicitation rules in these states. Consult the state's advertising regulations before building such audiences.
- 2. Specialty claims.** If the firm advertises as a "specialist" in a practice area, most state bars require formal specialty certification. Tracking events that imply specialization (e.g., `content_name: "Personal Injury Specialist Inquiry"`) can compound the issue.
- 3. Comparison and result-based advertising.** Cannot include specific case-result data ("won \$2.5M for client") in any tracked event. If your conversion-page URL contains result data, sanitize it before firing.

Event mapping for legal

Action	Event	Notes
Email captured (newsletter or guide download)	<code>Lead</code>	<code>content_name: "Resource Download"</code> , no practice area
Contact form submitted	<code>Contact</code>	Strip all free-text; no practice-area in <code>custom_data</code>
Free consultation requested	<code>Schedule</code>	<code>content_name: "Consultation"</code> , no specialty
Engagement letter signed (server-side only)	<code>CompleteRegistration</code>	Server-side fire from CRM, never browser

The "engagement letter signed" event is high-value but should never fire in the browser. Once a client has signed an engagement letter, attorney-client privilege attaches in full and tracking pixels are inappropriate. Fire this event server-side from the CRM with hashed email only.

Conflict-check forms

Many firms use a pre-consultation conflict-check form to make sure the prospect's matter does not conflict with existing client representations. The data captured (opposing party names, related companies, specific matter details) is sensitive. Two rules:

1. **Conflict-check forms must not fire any tracking events on submission.** Disable analytics, disable tags, treat the form submission as a private API call.
2. **The form data must be stored separately from marketing data.** Firms commonly store conflict-check data in the case management system (Clio, MyCase, PracticePanther) and never let it touch the marketing CRM or analytics warehouse.

Call tracking for legal

CallRail or similar can be used safely with these settings:

- **Disable call recording** unless the firm has a clear consent-and-retention policy
- **Or** enable recording with a state-specific whisper message ("This call may be recorded for quality purposes") and store recordings in a HIPAA-grade vault even though HIPAA does not apply, because the recording may capture privileged communication
- **Two-party consent states** (California, Florida, Illinois, Maryland, Massachusetts, Michigan, Montana, Nevada, New Hampshire, Pennsylvania, Washington) require explicit caller consent before recording. Build the workflow around this.

4.3 The Home Service Pattern (Trades, Contractors, Multi-Location Service Brands)

Lighter compliance constraints, but the attribution problem is acute. Home service businesses are the verticals where tracking failures cost the most because the CAC is high and the call-to-conversion ratio is high.

Architecture decision

Standard Pixel plus server-side CAPI both work. The vertical-specific layer is mostly about call tracking, service-area attribution, and capturing the source of every lead.

What to add

1. **CallRail with full DNI** on every page. Phone calls drive the majority of leads in this vertical and DNI lets you trace each call back to the campaign and keyword that drove it.

- 2. Service-area tracking** via custom dimensions in GA4. Capture the visitor's inferred service area (from IP geolocation or from their search query) and pass it as a custom dimension.
- 3. "How did you find us" form field** as a required dropdown on every contact form. The qualitative data is the only way to capture word-of-mouth and offline-source attribution.
- 4. Route tracking** for businesses with field operations. When a technician arrives on-site, tag that visit back to the original lead source for true closed-loop attribution.

Event mapping for home service

Action	Event	Notes
Phone call completed (over qualifying duration)	Lead	value: , content_name: "Phone Lead"
Quote form submitted	Lead	value: , content_name: "Quote Request"
Appointment booked	Schedule	value: , content_name: "Service Booking"
Service completed (server-side from CRM)	Purchase	value: , currency: USD

The "service completed" event is the closed-loop signal that ties the original ad click all the way to the actual revenue. This is what makes Smart Bidding actually smart for home service businesses.

Service-area schema and tracking

Home service businesses often operate across multiple ZIP codes or cities. Track this in two places:

In GA4 as a custom dimension:

```
gtag('event', 'page_view', {
  service_area: 'San Diego County',
  service_area_zip: '92103'
});
```

In Meta CAPI custom_data:

```
{
  "custom_data": {
    "value": 250.00,
    "currency": "USD",
    "content_name": "Quote Request",
    "content_category": "Plumbing",
    "service_area": "San Diego County"
  }
}
```

This data is not PII and is safe to send. It powers location-based retargeting and lets you verify your campaigns are reaching the right service areas.

Multi-location considerations

For franchise or multi-location home service brands, every location needs its own:

- **Google Business Profile** with consistent NAP
- **Local landing page** with `LocalBusiness` schema (see The Local Schema Pack for the schema specifics)
- **Dedicated CallRail tracking number pool** mapped to that location's GBP listing
- **GA4 location custom dimension** so you can segment performance by location

The companion git repo includes a `lib/locations/multi-location.ts` template for building this configuration consistently across locations.

Vertical comparison summary

Aspect	Doctor	Lawyer	Home Service
Browser-side Pixel	Remove	Use with care	Use freely
Server-side CAPI	Required, via BAA-CDP	Required for Lead events	Recommended
LDU flag	Always on	On for CA / EEA visitors	On for CA / EEA visitors
Free-text form data	Never to ad platforms	Never to ad platforms	OK if visitor consents
Call recording	BAA + locked access only	Disabled or vault-only	Standard, with whisper
Specialty / category in events	Never	Never	OK (e.g., "Plumbing")
Lookalike audiences	Hashed patient list, LDU on	Verify against state bar	OK
Closed-loop revenue tracking	Optional, server-side only	Optional, post-engagement	Mandatory for ROI

The compliance settings differ, but the same git repo (with the right vertical config file selected) supports all three.

This pattern is one piece of a wider toolkit. Adjacent playbooks at aibrandfactory.com.

Part 5, Call Tracking Plus Lead Attribution

This is where most professional services lose the most ad-spend efficiency. A visitor clicks a Google Ad, lands on the site, browses for two minutes, then calls the phone number. The CallRail dashboard records the call, but the ad campaign that generated the click gets zero credit because the click identifier (`gclid`) was never carried into the call record. Multiply by hundreds of calls per month and the ad bidding model is operating on garbage data.

This part is the fix: dynamic number insertion, first-party UTM capture, lead-attribution database design, and the Google Search Console API integration that pulls the actual organic queries driving traffic.

Dynamic Number Insertion (DNI) with CallRail

CallRail dynamic number insertion replaces the static phone number on a page with a campaign-specific tracking number when the visitor lands. Every visitor sees a unique number, the call goes to the same destination, but CallRail logs which campaign and which keyword drove the call.

The script loads from CallRail's CDN and is initialized with the practice's company token:

```
<!-- Static HTML pattern -->
<script
  src="https://cdn.callrail.com/companies/{COMPANY_TOKEN}/tracking.js"
  async>
</script>
```

For React and Next.js sites, three issues come up that the static script does not have:

Issue 1, React renders phone numbers after the DNI script runs

The CallRail script scans the DOM for phone numbers when it loads. If the phone number is rendered by a React component that hydrates after the script runs, DNI misses it.

The fix: mark the phone number with a `data-callrail-number` attribute. CallRail's MutationObserver watches for elements with this attribute and swaps them when they appear in the DOM, regardless of when they hydrate.

```
function PhoneNumber({ display, target }) {
  return (
    <a
      href={`tel:${target}`}
      data-callrail-number={target}
    >
      {display}
    </a>
  );
}
```

Issue 2, SPA navigation does not trigger DNI re-scan

When the visitor navigates between pages in a Next.js or React SPA, the DOM changes but the DNI script does not automatically re-scan. The new page's phone numbers stay static.

The fix: trigger a manual rescan in the route-change effect.

```
import { usePathname } from 'next/navigation';
import { useEffect } from 'react';

function CallRailRescan() {
  const pathname = usePathname();

  useEffect(() => {
    if (typeof window !== 'undefined' && window.CallRail) {
      window.CallRail.scan();
    }
  }, [pathname]);

  return null;
}
```

Issue 3, Phone numbers in images cannot be swapped

DNI cannot swap phone numbers rendered as part of an image (logo banners, hero images, infographics). Always render phone numbers as text. If the design requires a styled appearance, use CSS, not bitmap text.

CallRail event integration with GA4 and Google Ads

CallRail does not auto-fire conversion events to GA4 or Google Ads. The integration has to be built explicitly.

For GA4, listen for the `cr-call-completed` browser event:

```
window.addEventListener('cr-call-completed', (event) => {
  const callDetails = event.detail;

  // Only count qualified calls (over a duration threshold)
  if (callDetails.duration_seconds >= 60) {
    gtag('event', 'qualified_call', {
      campaign: callDetails.cnam_caller_name,
      duration: callDetails.duration_seconds,
      tracking_number: callDetails.tracking_phone_number
    });
  }
});
```

For Google Ads, configure the integration in CallRail's account settings. CallRail will then upload calls as offline conversions via Google's offline conversion import API. No additional code needed if the integration toggle is on.

First-party UTM capture middleware

The single most important attribution upgrade for any professional services site. Capture every UTM parameter, click identifier, and referrer on the visitor's first page load and persist it as a first-party cookie or in localStorage. Then attach it to every form submission, every CallRail call, and every server-side conversion event.

The middleware pattern (Next.js 16 App Router):

```

import { NextRequest, NextResponse } from 'next/server';

export function middleware(request: NextRequest) {
  const response = NextResponse.next();

  const existing = request.cookies.get('visit_data');
  if (existing) return response;

  const url = request.nextUrl;

  const visitData = {
    referrer: request.headers.get('referer') || '',
    landing_page: url.pathname,
    timestamp: new Date().toISOString(),
    gclid: url.searchParams.get('gclid'),
    mscldid: url.searchParams.get('msclid'),
    fbclid: url.searchParams.get('fbclid'),
    ttclid: url.searchParams.get('ttclid'),
    utm_source: url.searchParams.get('utm_source'),
    utm_medium: url.searchParams.get('utm_medium'),
    utm_campaign: url.searchParams.get('utm_campaign'),
    utm_term: url.searchParams.get('utm_term'),
    utm_content: url.searchParams.get('utm_content')
  };

  response.cookies.set('visit_data', JSON.stringify(visitData), {
    maxAge: 60 * 60 * 24 * 30, // 30 days
    httpOnly: false, // needs to be readable by client JS for form attachment
    sameSite: 'lax',
    path: '/'
  });

  return response;
}

export const config = {
  matcher: '/((?!api|_next/static|_next/image|favicon.ico).*)'
};

```

For non-Next sites, the same pattern applies in any server-rendered framework or via an Edge Worker.

Hidden form fields

Every conversion form on the site needs hidden fields populated from `visit_data` :

```
function ContactForm() {
  const [tracking, setTracking] = useState({});

  useEffect(() => {
    const cookie = document.cookie
      .split('; ')
      .find(row => row.startsWith('visit_data='));
    if (cookie) {
      try {
        setTracking(JSON.parse(decodeURIComponent(cookie.split('=')[1])));
      } catch (e) {}
    }
  }, []);

  return (
    <form action="/api/lead" method="POST">
      <input type="email" name="email" required />
      <input type="tel" name="phone" required />
      <textarea name="message" required />

      {/* Hidden attribution fields */}
      <input type="hidden" name="gclid" value={tracking.gclid || ''} />
      <input type="hidden" name="msclkid" value={tracking.msclkid || ''} />
      <input type="hidden" name="fbclid" value={tracking.fbclid || ''} />
      <input type="hidden" name="utm_source" value={tracking.utm_source || ''} />
      <input type="hidden" name="utm_medium" value={tracking.utm_medium || ''} />
      <input type="hidden" name="utm_campaign" value={tracking.utm_campaign || ''} />
      <input type="hidden" name="utm_term" value={tracking.utm_term || ''} />
      <input type="hidden" name="referrer" value={tracking.referrer || ''} />
      <input type="hidden" name="landing_page" value={tracking.landing_page || ''} />

      <button type="submit">Send</button>
    </form>
  );
}
```

When the form submits, the server-side handler stores the lead with all attribution intact. The Meta CAPI handler reads the `fbclid` and includes it as `fbclid` . The Google Ads Enhanced Conversions handler reads the `gclid` and uses it in the `gclid_date_time_pair` .

The lead_attribution table

For practices that build on Postgres, MySQL, or any relational database, the recommended table schema for storing complete lead attribution:

```

CREATE TABLE lead_attribution (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id     UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),

  -- First-touch attribution
  first_referrer    TEXT,
  first_landing_page TEXT,
  first_utm_source  TEXT,
  first_utm_medium  TEXT,
  first_utm_campaign TEXT,
  first_utm_term    TEXT,
  first_utm_content TEXT,
  first_gclid       TEXT,
  first_msclkid     TEXT,
  first_fbclid      TEXT,
  first_ttclid      TEXT,
  first_visit_at    TIMESTAMPTZ,

  -- Last-touch attribution (the visit that converted)
  last_referrer     TEXT,
  last_landing_page TEXT,
  last_utm_source   TEXT,
  last_utm_medium   TEXT,
  last_utm_campaign TEXT,
  last_utm_term     TEXT,
  last_gclid        TEXT,

  -- Qualitative
  how_did_you_hear TEXT,    -- from the form dropdown

  -- Visitor signals
  ip_country        TEXT,
  ip_region         TEXT,
  ip_city           TEXT,
  user_agent        TEXT,
  device_type       TEXT
);

CREATE INDEX idx_lead_attribution_lead_id ON lead_attribution(lead_id);
CREATE INDEX idx_lead_attribution_first_utm_source ON lead_attribution(first_utm_source);
CREATE INDEX idx_lead_attribution_first_utm_campaign ON
lead_attribution(first_utm_campaign);

```

This single table makes possible reports the practice could not build before:

- Cost per lead by campaign, with first-touch and last-touch comparisons
- Which organic landing pages convert to leads (vs. just driving traffic)
- The gap between self-reported source ("I found you on Google") and actual source (Meta retargeting)
- Geo-distribution of leads by service area

Google Search Console API: pulling actual organic queries

GA4 will not show the actual search queries that drove organic traffic. That data lives in Google Search Console and has to be pulled via the Search Console API.

The endpoint and a working query for the last 30 days of top queries by landing page:

```
const { google } = require('googleapis');

const auth = new google.auth.GoogleAuth({
  keyFile: process.env.GOOGLE_SERVICE_ACCOUNT_KEY_PATH,
  scopes: ['https://www.googleapis.com/auth/webmasters.readonly']
});

const searchconsole = google.searchconsole({ version: 'v1', auth });

async function getOrganicQueries(siteUrl, startDate, endDate) {
  const response = await searchconsole.searchanalytics.query({
    siteUrl,
    requestBody: {
      startDate,
      endDate,
      dimensions: ['query', 'page'],
      rowLimit: 25000,
      dataState: 'all'
    }
  });

  return response.data.rows || [];
}
```

Run this nightly via a cron job and store the results in your data warehouse. You will see:

- Which actual queries (not "not provided") drive each landing page

- The CTR for each query, which guides title tag and meta description rewrites
- Position changes over time, which flags ranking decay before it shows in traffic

For professional services that publish content, this is the closest you can get to knowing what your audience actually searches.

The "How did you find us" form field

The qualitative ground truth that no tracking pixel can capture. A single dropdown on every contact form:

How did you hear about us?

- Google search
- Bing or other search
- Social media (Facebook, Instagram, TikTok, LinkedIn)
- YouTube
- Friend or family referral
- Existing patient / client referral
- Drove by, saw the sign
- Mailer or print ad
- Other (please specify)

This data tells you what the analytics never can: word-of-mouth volume, drive-by visibility for a physical location, the success of an offline campaign, the post-purchase referral rate. Compare this to the auto-tracked attribution and the gap is often surprising. Many practices learn that 30 to 50 percent of their best leads come from word-of-mouth that none of their tracking captures.

The complete attribution flow

When all of the above is in place, a single lead generates this much data:

1. Visitor lands on /services/consultation with ?

`utm_source=google&utm_medium=cpc&gclid=ABC123`

2. Middleware sets `visit_data` cookie with all params + referrer + landing page + timestamp

3. **Visitor browses for 4 minutes**, GA4 captures sessions and engagement (consent permitting)
4. **Visitor calls the CallRail tracking number** displayed on the page
5. **CallRail logs the call** with the tracking number's source mapping
6. **Server-side webhook from CallRail** ties the call to the visitor's session via the IP + tracking number pool match
7. **cr-call-completed event fires** in the browser, GA4 logs a qualified_call event
8. **CallRail uploads call as Google Ads conversion** via the integration
9. **Lead enters CRM** with full first-touch attribution including `gclid`
10. **Server-side handler fires Meta CAPI Lead event** with hashed email + `fbclid`
11. **Server-side handler fires Google Ads Enhanced Conversion** with hashed email + `gclid`
12. **Lead converts to client / patient / appointment**, server-side fires Purchase event with revenue
13. **Smart Bidding model now has the closed-loop signal** to bid more aggressively on similar visitors

The companion git repo includes the full middleware, the form components, the API route handlers for Meta and Google Ads, the SQL migrations for the lead_attribution table, and the GSC API integration in `lib/gsc/queries.ts`.

Part 6, Validation, Deployment, and the 8 Failures That Silently Break Tracking

This part is the operational layer. How to verify every component is firing correctly, how to deploy the stack into a production site, what to monitor in the first 8 weeks, and the eight failure patterns that silently kill tracking.

Validation tools (run in this order before going live)

1, GTM Preview Mode

Open Tag Manager, click Preview, enter your site URL. A new browser tab opens with a debug console attached. As you click through the site, GTM shows every tag that fires, what trigger fired it, and the dataLayer state at the time. Use this to verify:

- The Consent Mode default fires before any other tag
- GA4 page_view fires only after consent is granted
- Meta Pixel does NOT fire if marketing consent is denied
- CallRail script loads after consent

This is the single most useful tool. If a tag is broken, GTM Preview shows it.

2, Google Tag Assistant (Chrome extension)

Install Tag Assistant Companion. It sits in the Chrome toolbar and validates GA4, Google Ads, and Floodlight tags on every page load. It catches:

- Duplicate tags (the same GA4 tag firing twice on a page)
- Missing required parameters
- Misconfigured Consent Mode signals
- Tags loading before consent default

3, Meta Events Manager Test Events

In Meta Ads Manager, go to Events Manager > Data Sources > [Your Pixel] > Test Events. Enter your site URL. Trigger a conversion (submit a form, click the call button). The Test Events panel shows every event Meta receives in real time, both from the Pixel and from CAPI. Use this to verify:

- Pixel and CAPI both arrive for the same conversion (deduplication working)
- `event_id` matches between Pixel and CAPI
- `user_data` fields are populated and hashed
- No PHI or restricted data is sneaking through

4, Google Ads Conversion Diagnostics

In Google Ads, Tools > Measurement > Conversions > [Your Conversion Action] > Diagnostics. Shows the last 7 days of conversions, the source (browser tag vs Enhanced Conversions API), and the match rate. A healthy match rate is 60 percent or higher; below that, your hashing is wrong or your `gclid` capture is missing.

5, CallRail's manual DNI test

In CallRail dashboard > Settings > Trackers > [Your Tracker] > Test. Enter the page URL. CallRail loads the page in a sandbox and reports which phone numbers were swapped, which were not, and why. Run this on every template the site has (homepage, service pages, contact, blog).

6, Manual JSON-LD parse test (for the schema layer that Part 5 references)

```
JSON.parse(document.querySelector('script[type="application/ld+json"]').textContent)
```

If this throws, the schema is malformed. Crawlers will skip it.

Deployment checklist (in order)

A safe deployment sequence for the full stack:

1. **Set up GTM container** with the placeholder tags (GA4, Google Ads, Meta Pixel) but trigger them on a non-production trigger (e.g., a page that does not exist yet)
2. **Set up the CMP** (vanilla-cookieconsent or equivalent) and verify it loads correctly with all categories defaulted to denied
3. **Configure Consent Mode v2 default** in the page template
4. **Wire CMP consent updates** to fire `gtag('consent', 'update', ...)` on accept
5. **Activate the GA4 tag in GTM** with the consent-aware trigger
6. **Verify in GTM Preview** that the full consent flow works
7. **Deploy the middleware** for first-party UTM capture
8. **Update conversion forms** with hidden tracking fields
9. **Build the server-side API route handlers** (Meta CAPI, Google Ads Enhanced)
10. **Test conversion fires** end-to-end via Meta Test Events and Google Ads Diagnostics
11. **Activate the Meta Pixel tag in GTM** with consent-aware trigger
12. **Set up CallRail** with company token, install via GTM with consent trigger
13. **Test DNI** on every page template
14. **Verify call events** flow to GA4 and Google Ads
15. **Set up the lead_attribution table** in the database
16. **Configure GSC API** with service account credentials
17. **Schedule the GSC nightly sync** via cron

18. Enable email notifications for any tag errors in GTM, GA4, and Meta Events Manager

This sequence avoids the trap of activating a tag before its dependencies are ready.

Monitoring in the first 8 weeks

Three signals to watch weekly:

- 1. GA4 > Configure > DebugView:** shows real-time events from any session you open in debug mode. Use this to spot regressions when the dev team ships changes.
- 2. Meta Events Manager > Diagnostics:** surfaces any data-quality warnings (e.g., low match quality, missing fields, high deduplication failures).
- 3. Google Ads > Conversions > Diagnostics:** shows match rate trend and conversion volume trend; sudden drops are usually a tag misconfiguration.

If any of the three flags an issue, fix it that week. Tag debt compounds.

The 8 failures that silently break tracking

Failure 1, Consent Mode v2 not initialized before tag firing

The GA4 or Meta Pixel tag fires before `gtag('consent', 'default', ...)` runs. The tag captures one or more events with default-granted consent before being told to deny. Fix: place the consent default script as the very first script in the page template, with `strategy="beforeInteractive"` in Next.js or before any other tag in static HTML.

Failure 2, Meta Pixel firing without consent gating

The Pixel loads via a static